

# 1. Problem Selection and Motivation

## Problem Statement

As a college student, I frequently need to track my grades across multiple courses, calculate my GPA, and understand my academic performance. Manual tracking using spreadsheets or paper is error-prone, time-consuming, and lacks interactivity. A simple, console-based grade management system would solve this real-world problem.

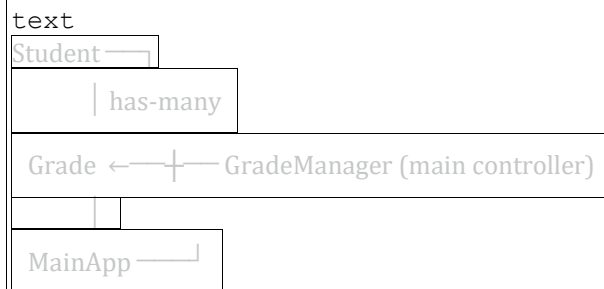
## Why Java Platform?

- **Course Relevance:** This project directly applies concepts from the "Programming in Java" course: OOP (classes, objects), collections (`ArrayList`, `Scanner`), file I/O (`PrintWriter`), loops, conditionals, and exception handling.
- **Platform Independence:** Java runs on any machine with JDK, making it perfect for sharing with professors and classmates.
- **Strong OOP Support:** Java's class-based structure perfectly fits modeling students, grades, and managers as separate objects.
- **Console-Based:** No GUI frameworks needed, focusing on core programming skills.

# 2. System Design and Architecture

The project uses **Object-Oriented Design** with three main classes:

## Class Diagram



## Key Design Decisions

1. **Encapsulation:** Private fields with public getters/setters.
2. **Composition:** `Student` contains `List<Grade>` objects.

### 3. Separation of Concerns:

- `Student` handles individual GPA calculation.
- `Grade` handles grade conversion (percentage → letter grade → GPA points).
- `GradeManager` handles business logic, user interaction, and persistence.

## 3. Code Explanation

### 3.1 Student Class

```
java
class Student {
    private String studentId, name, email;
    private List<Grade> grades;
    private static int idCounter = 1000;
```

- **Auto-ID Generation:** `STU1001`, `STU1002`... using `static` counter.
- **GPA Calculation:** Weighted average using `for-each` loop:

```
java
totalPoints += grade.getGradePoints() * grade.getCredits();
return totalPoints / totalCredits;
```

### 3.2 Grade Class

```
java
class Grade {
    private double score; // 0-100
```

- **Input Validation:** `Math.max(0, Math.min(100, score))` clamps scores.
- **Grade Conversion:**

```
text
90+ → A (4.0 pts), 80-89 → B (3.0 pts), etc.
```

### 3.3 GradeManager Class (Core Logic)

- **Menu-Driven Interface:** `switch-case` with 6 options.
- **Stream API:** Modern Java for finding students:

```
java
students.stream().filter(s -> s.getStudentId().equalsIgnoreCase(id)).findFirst()
```

- **File Persistence:** Custom CSV format using `PrintWriter`:

```
text
STUDENT|STU1001|Alice|alice@university.edu
GRADE|CS101|Intro to Programming|92.5|3|Fall 2025
```

### 3.4 Main Class

- Loads **demo data** to showcase functionality.
- Calls `GradeManager.runInteractiveMenu()` for user interaction.

## 4. Key Course Concepts Applied

| Java Concept             | Where Used                                                                  |
|--------------------------|-----------------------------------------------------------------------------|
| Classes/OOP              | <code>Student</code> , <code>Grade</code> , <code>GradeManager</code>       |
| Collections              | <code>ArrayList&lt;Student&gt;</code> , <code>ArrayList&lt;Grade&gt;</code> |
| Inheritance/Polymorphism | <code>toString()</code> override                                            |
| Exception Handling       | <code>try-catch</code> in <code>saveToFile()</code>                         |
| File I/O                 | <code>PrintWriter</code> , <code>FileWriter</code>                          |
| Control Structures       | <code>while</code> , <code>switch</code> , <code>for-each</code>            |
| String Formatting        | <code>String.format()</code> , <code>printf()</code>                        |

## 5. Sample Output

```
text
|
```

|                                            |               |       |       |         |           |  |  |  |  |
|--------------------------------------------|---------------|-------|-------|---------|-----------|--|--|--|--|
| STUDENT GRADE TRACKER                      |               |       |       |         |           |  |  |  |  |
|                                            |               |       |       |         |           |  |  |  |  |
| 1. Add Student 2. Add Grade 3. View Report |               |       |       |         |           |  |  |  |  |
| ...                                        |               |       |       |         |           |  |  |  |  |
| STUDENT ACADEMIC REPORT                    |               |       |       |         |           |  |  |  |  |
| Student ID: STU1001 Name: Alice Johnson    |               |       |       |         |           |  |  |  |  |
| Overall GPA: 3.67                          |               |       |       |         |           |  |  |  |  |
| Code                                       | Course Name   | Score | Grade | Credits | Semester  |  |  |  |  |
| CS101                                      | Intro to Prog | 92.5  | A     | 3       | Fall 2025 |  |  |  |  |

---

## 6. Challenges Faced

- **Console Formatting:** Achieving aligned table output using `printf` and `repeat()`.
- **Input Validation:** Handling invalid scores and student IDs.
- **File Format Design:** Creating a simple, readable CSV format.